

📄 PROJET DATA FULL-STACK

Pipeline RAW → ENRICHED → SQL → DASHBOARD

Niveau : Bac +4 – Développement **Travail** : Individuel **Stack** : Libre (langages, frameworks, bases de données au choix)

📄 Objectif général du projet

Ce projet a pour objectif de vous placer dans un **contexte professionnel réaliste**, proche de ce que l'on retrouve dans les équipes data / produit en entreprise.

Vous devrez concevoir **une chaîne complète de traitement de données**, depuis la collecte jusqu'à l'affichage dans un dashboard, en respectant les bonnes pratiques suivantes :

- séparation claire des responsabilités
- conservation de la donnée brute
- enrichissement maîtrisé
- modélisation SQL exploitable
- API performante
- interface fluide
- qualité logicielle via les tests

Ce projet évalue votre capacité à **raisonner architecture, data, produit et qualité**.

📄 Choix du sujet (OBLIGATOIRE)

Vous devez choisir **UN SEUL sujet** parmi les trois proposés ci-dessous. Le choix du sujet influence les données collectées, les enrichissements et le dashboard.

📄 Sujet 1 — Offres d'emploi Tech & tendances du marché

Problématique

Analyser le marché de l'emploi dans le secteur technologique afin de comprendre :

- quelles technologies sont les plus demandées
- les niveaux de salaire
- la répartition géographique
- la part de télétravail

Sources de données possibles

- API open-data
- pages publiques d'offres d'emploi (scraping simple, sans login)
- dataset public (CSV ou JSON)

Exemple de donnée brute

```
{
  "title": "Backend Developer",
  "company": "TechCorp",
  "location": "Paris",
  "description": "Node.js, PostgreSQL, Docker",
  "salary": "50k-60k",
  "posted_at": "2025-01-10"
}
```

📄 Sujet 2 — Événements culturels & attractivité territoriale

Problématique

Analyser l'offre culturelle afin de comprendre :

- les types d'événements les plus fréquents

- leur localisation géographique
- les périodes les plus actives

Sources de données possibles

- open-data municipal ou national
- API publique d'événements culturels

Exemple de donnée brute

```
{
  "name": "Concert Jazz",
  "city": "Lyon",
  "address": "12 rue des Arts",
  "category": "Music",
  "date": "2025-02-20"
}
```

☒ Sujet 3 — Produits alimentaires & qualité nutritionnelle

Problématique

Analyser la qualité des produits alimentaires afin d'identifier :

- les catégories dominantes
- les marques principales
- les niveaux de qualité nutritionnelle

Sources de données possibles

- OpenFoodFacts
- open-data alimentaire

Exemple de donnée brute

```
{
  "product_name": "Cereal X",
  "brand": "BrandY",
  "nutriscore": "C",
  "ingredients": "...",
  "category": "Breakfast"
}
```

☒ Architecture globale attendue

Votre projet doit respecter la logique suivante :

```
Collecte
  ↓
MongoDB (RAW)
  ↓
Enrichissement via API
  ↓
MongoDB (ENRICHED)
  ↓
ETL
  ↓
Base SQL
  ↓
API Backend
  ↓
Dashboard
```

Chaque bloc a un rôle précis et ne doit pas mélanger les responsabilités.

🔖 ÉTAPE 1 — Collecte des données

Objectif

Récupérer des données **réelles, complètes** et **non modifiées**.

Pourquoi cette étape est importante ?

- En entreprise, la donnée brute est une référence
- Elle permet de rejouer le pipeline
- Elle garantit la traçabilité des sources

Attendus

- Script ou service de collecte (CLI, cron, endpoint, etc.)
- Minimum **300 entrées**
- Gestion basique des erreurs (timeout, format invalide, données manquantes)

🔖 ÉTAPE 2 — Stockage brut (MongoDB RAW)

Objectif

Stocker **exactement ce qui est reçu**, sans aucune transformation.

Pourquoi MongoDB ?

- Structure flexible
- Support natif du JSON
- Très adapté à la donnée brute et hétérogène

Structure obligatoire d'un document RAW

```
{
  "_id": "...",
  "source": "nom_de_la_source",
  "fetched_at": "2025-01-15T10:30:00Z",
  "raw_hash": "hash_unique",
  "payload": { "donnée brute complète" }
}
```

Règles importantes

- `payload` contient 100 % de la donnée collectée
- Aucun traitement métier à ce niveau

- Le `raw_hash` permet d'éviter les doublons
- La collection RAW ne doit jamais être modifiée

🔖 ÉTAPE 3 — Enrichissement des données

Objectif

Transformer la donnée brute en donnée **utile** pour l'analyse et le produit.

Pourquoi séparer l'enrichissement ?

- Les API peuvent tomber
- Les règles métiers évoluent
- Le brut reste inchangé

Enrichissements requis

Vous devez implémenter **au minimum 2 enrichissements**, par exemple :

- géocodage (adresse → latitude / longitude)
- extraction de compétences depuis un texte
- normalisation de salaire ou de dates
- catégorisation automatique
- calcul d'un score interne

Exemple de document enrichi

```
{
  "raw_id": "...",
  "status": "success",
  "enriched_at": "2025-01-15T11:00:00Z",
  "data": {
    "skills": ["node", "sql"],
    "salary_year": 55000,
    "remote": true
  }
}
```

Gestion des erreurs

- statuts : `pending`, `success`, `failed`
- gestion simple des retries
- stockage de l'erreur (code, message)

🔖 ÉTAPE 4 — Base SQL (données exploitables)

Objectif

Créer une base relationnelle **optimisée pour l'exploitation et le dashboard**.

Pourquoi SQL ?

- jointures efficaces
- index
- agrégations
- performances élevées

Attendus

- schéma relationnel cohérent
 - clés primaires et étrangères
 - index sur les colonnes filtrées
 - contraintes (NOT NULL, UNIQUE, etc.)
-

📄 ETL — MongoDB → SQL

Objectif

Transférer les données enrichies vers la base SQL.

Contraintes

- pas de doublons
- mapping clair Mongo → SQL
- script rejouable (idempotent)

Pourquoi cette étape ?

Elle sépare clairement :

- la collecte
- la transformation
- l'exploitation produit

📄 ÉTAPE 5 — API Backend

Objectif

Exposer les données SQL au dashboard.

Endpoints minimum attendus

- GET /items : liste paginée avec filtres
- GET /items/{id} : détail d'un élément
- GET /stats : statistiques globales

Exemple d'appel

```
GET /items?category=tech&minSalary=40000&page=1&pageSize=20
```

Contraintes

- pagination serveur obligatoire
- filtres performants
- réponses rapides

📄 ÉTAPE 6 — Dashboard

Objectif

Afficher les données **de manière fluide, claire et exploitable**.

Écrans minimum

1. Liste paginée
2. Vue détail
3. Bloc statistiques

Contraintes

- minimum 4 filtres pertinents
- pas de chargement massif de données
- interface claire et lisible

📄 ÉTAPE 7 — Tests

Tests unitaires (obligatoires)

- parsing des données
- enrichissement
- normalisation
- mapping ETL

Tests d'intégration (obligatoires)

- appel réel d'un endpoint API
 - requête SQL
 - pipeline complet sur un petit jeu de données
-

📦 Livrables attendus

Vous devez fournir :

- un dépôt GitHub
 - un README détaillé :
 - instructions d'installation
 - description de l'architecture
 - choix techniques
 - limites du projet
 - le code source
 - les tests
 - le schéma SQL (migration ou fichier SQL)
-

📦 Résultat attendu

Un **mini-produit data professionnel**, structuré, testable, et présentable dans un contexte d'entreprise.